JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Proxy

java.lang.Object
    java.lang.reflect.Proxy

All Implemented Interfaces:

Serializable

```
public class Proxy
extends Object
implements Serializable
```

`Proxy` provides static methods for creating objects that act like instances of interfaces but allow for customized method invocation. To create a proxy instance for some interface `Foo`:

```
    InvocationHandler handler = new MyInvocationHandler(...);
    Foo f = (Foo) Proxy.newProxyInstance(Foo.class.getClassLoader(),
                                         new Class<?>[] { Foo.class },
                                         handler);
```

A *proxy class* is a class created at runtime that implements a specified list of interfaces, known as *proxy interfaces*. A *proxy instance* is an instance of a proxy class. Each proxy instance has an associated *invocation handler* object, which implements the interface `InvocationHandler`. A method invocation on a proxy instance through one of its proxy interfaces will be dispatched to the `invoke` method of the instance's invocation handler, passing the proxy instance, a `java.lang.reflect.Method` object identifying the method that was invoked, and an array of type `Object` containing the arguments. The invocation handler processes the encoded method invocation as appropriate and the result that it returns will be returned as the result of the method invocation on the proxy instance.

A proxy class has the following properties:

- The unqualified name of a proxy class is unspecified. The space of class names that begin with the string `"$Proxy"` should be, however, reserved for proxy classes.
- The package and module in which a proxy class is defined is specified below.
- A proxy class is *final and non-abstract*.
- A proxy class extends `java.lang.reflect.Proxy`.
- A proxy class implements exactly the interfaces specified at its creation, in the same order. Invoking `getInterfaces` on its `Class` object will return an array containing the same list of interfaces (in the order specified at its creation), invoking `getMethods` on its `Class` object will return an array

of `Method` objects that include all of the methods in those interfaces, and invoking `getMethod` will find methods in the proxy interfaces as would be expected.
- The `ProtectionDomain` of a proxy class is the same as that of system classes loaded by the bootstrap class loader, such as `java.lang.Object`.
- The `Proxy.isProxyClass` method can be used to determine if a given class is a proxy class.

A proxy instance has the following properties:

- Given a proxy instance `proxy` and one of the interfaces, `Foo`, implemented by its proxy class, the following expression will return true:

```
    proxy instanceof Foo
```

  and the following cast operation will succeed (rather than throwing a `ClassCastException`):

```
    (Foo) proxy
```

- Each proxy instance has an associated invocation handler, the one that was passed to its constructor. The static `Proxy.getInvocationHandler` method will return the invocation handler associated with the proxy instance passed as its argument.
- An interface method invocation on a proxy instance will be encoded and dispatched to the invocation handler's `invoke` method as described in the documentation for that method.
- A proxy interface may define a default method or inherit a default method from its superinterface directly or indirectly. An invocation handler can invoke a default method of a proxy interface by calling `InvocationHandler::invokeDefault`.
- An invocation of the `hashCode`, `equals`, or `toString` methods declared in `java.lang.Object` on a proxy instance will be encoded and dispatched to the invocation handler's `invoke` method in the same manner as interface method invocations are encoded and dispatched, as described above. The declaring class of the `Method` object passed to `invoke` will be `java.lang.Object`. Other public methods of a proxy instance inherited from `java.lang.Object` are not overridden by a proxy class, so invocations of those methods behave like they do for instances of `java.lang.Object`.

Package and Module Membership of Proxy Class

The package and module to which a proxy class belongs are chosen such that the accessibility of the proxy class is in line with the accessibility of the proxy

interfaces. Specifically, the package and the module membership of a proxy class defined via the `getProxyClass(ClassLoader, Class[])` or `newProxyInstance(ClassLoader, Class[], InvocationHandler)` methods is specified as follows:

1. If all the proxy interfaces are in *exported* or *open* packages:
    1. if all the proxy interfaces are *public*, then the proxy class is *public* in an unconditionally exported but non-open package. The name of the package and the module are unspecified.
    2. if at least one of all the proxy interfaces is *non-public*, then the proxy class is *non-public* in the package and module of the non-public interfaces. All the non-public interfaces must be in the same package and module; otherwise, proxying them is not possible.
2. If at least one proxy interface is in a package that is *non-exported* and *non-open*:
    1. if all the proxy interfaces are *public*, then the proxy class is *public* in a *non-exported*, *non-open* package of *dynamic module*. The names of the package and the module are unspecified.
    2. if at least one of all the proxy interfaces is *non-public*, then the proxy class is *non-public* in the package and module of the non-public interfaces. All the non-public interfaces must be in the same package and module; otherwise, proxying them is not possible.

Note that if proxy interfaces with a mix of accessibilities -- for example, an exported public interface and a non-exported non-public interface -- are proxied by the same instance, then the proxy class's accessibility is governed by the least accessible proxy interface.

Note that it is possible for arbitrary code to obtain access to a proxy class in an open package with `setAccessible`, whereas a proxy class in a non-open package is never accessible to code outside the module of the proxy class.

Throughout this specification, a "non-exported package" refers to a package that is not exported to all modules, and a "non-open package" refers to a package that is not open to all modules. Specifically, these terms refer to a package that either is not exported/open by its containing module or is exported/open in a qualified fashion by its containing module.

Dynamic Modules

A dynamic module is a named module generated at runtime. A proxy class defined in a dynamic module is encapsulated and not accessible to any module. Calling `Constructor.newInstance(Object...)` on a proxy class in a dynamic module will throw `IllegalAccessException`; `Proxy.newProxyInstance` method should be used instead.

A dynamic module can read the modules of all of the superinterfaces of a proxy class and the modules of the classes and interfaces referenced by all public method signatures of a proxy class. If a superinterface or a referenced class or interface, say `T`, is in a non-exported package, the module of `T` is updated to export the package of `T` to the dynamic module.

Methods Duplicated in Multiple Proxy Interfaces

When two or more proxy interfaces contain a method with the same name and parameter signature, the order of the proxy class's interfaces becomes significant. When such a *duplicate method* is invoked on a proxy instance, the `Method` object passed to the invocation handler will not necessarily be the one whose declaring class is assignable from the reference type of the interface that the proxy's method was invoked through. This limitation exists because the corresponding method implementation in the generated proxy class cannot determine which interface it was invoked through. Therefore, when a duplicate method is invoked on a proxy instance, the `Method` object for the method in the foremost interface that contains the method (either directly or inherited through a superinterface) in the proxy class's list of interfaces is passed to the invocation handler's `invoke` method, regardless of the reference type through which the method invocation occurred.

If a proxy interface contains a method with the same name and parameter signature as the `hashCode`, `equals`, or `toString` methods of `java.lang.Object`, when such a method is invoked on a proxy instance, the `Method` object passed to the invocation handler will have `java.lang.Object` as its declaring class. In other words, the public, non-final methods of `java.lang.Object` logically precede all of the proxy interfaces for the determination of which `Method` object to pass to the invocation handler.

Note also that when a duplicate method is dispatched to an invocation handler, the `invoke` method may only throw checked exception types that are assignable to one of the exception types in the `throws` clause of the method in *all* of the proxy interfaces that it can be invoked through. If the `invoke` method throws a checked exception that is not assignable to any of the exception types declared by the method in one of the proxy interfaces that it can be invoked through, then an unchecked `UndeclaredThrowableException` will be thrown by the invocation on the proxy instance. This restriction means that not all of the exception types returned by invoking `getExceptionTypes` on the `Method` object passed to the `invoke` method can necessarily be thrown successfully by the `invoke` method.

Since:

1.3

See Also:

InvocationHandler, Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
protected InvocationHandler	h	the invocation handler for this proxy instance.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	Proxy (InvocationHandler h)	Constructs a new Proxy instance from a subclass (typically, a dynamic proxy class) with the specified value for its invocation handler.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
static InvocationHandler	getInvocationHandler (Object proxy)	Returns the invocation handler for the specified proxy instance.
static Class<?>	getProxyClass (ClassLoader loader, Class<?>... interfaces)	**Deprecated.** Proxy classes generated in a named module are encapsulated and not accessible to code outside its module.
static boolean	isProxyClass(Class<?> cl)	Returns true if the given class is a proxy class.
static Object		Returns a proxy instance for the specified

	newProxyInstance(ClassLoader loader, Class<?>[] interfaces, InvocationHandler h)	interfaces that dispatches method invocations to the specified invocation handler.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

h

```
protected InvocationHandler h
```

the invocation handler for this proxy instance.

Constructor Details

Proxy

```
protected Proxy(InvocationHandler h)
```

Constructs a new `Proxy` instance from a subclass (typically, a dynamic proxy class) with the specified value for its invocation handler.

Parameters:

`h` - the invocation handler for this proxy instance

Throws:

`NullPointerException` - if the given invocation handler, `h`, is `null`.

Method Details

getProxyClass

```
@Deprecated
public static Class<?> getProxyClass(ClassLoader loader,
                                     Class<?>... interfaces)
                              throws IllegalArgumentException
```

Deprecated.
Proxy classes generated in a named module are encapsulated and not accessible to code outside its module. `Constructor.newInstance` will throw `IllegalAccessException` when it is called on an inaccessible proxy class. Use `newProxyInstance(ClassLoader, Class[], InvocationHandler)` to create a proxy instance instead.

Returns the `java.lang.Class` object for a proxy class given a class loader and an array of interfaces. The proxy class will be defined by the specified class loader and will implement all of the supplied interfaces. If any of the given interfaces is non-public, the proxy class will be non-public. If a proxy class for the same permutation of interfaces has already been defined by the class loader, then the existing proxy class will be returned; otherwise, a proxy class for those interfaces will be generated dynamically and defined by the class loader.

Parameters:

`loader` - the class loader to define the proxy class

`interfaces` - the list of interfaces for the proxy class to implement

Returns:

a proxy class that is defined in the specified class loader and that implements the specified interfaces

Throws:

`IllegalArgumentException` - if any of the restrictions on the parameters are violated

`NullPointerException` - if the `interfaces` array argument or any of its elements are `null`

See Also:

Package and Module Membership of Proxy Class

newProxyInstance

```
public static Object newProxyInstance(ClassLoader loader,
                                       Class<?>[] interfaces,
                                       InvocationHandler h)
```

Returns a proxy instance for the specified interfaces that dispatches method invocations to the specified invocation handler.

`IllegalArgumentException` will be thrown if any of the following restrictions is violated:

- All of `Class` objects in the given `interfaces` array must represent non-hidden and non-sealed interfaces, not classes or primitive types.

- No two elements in the `interfaces` array may refer to identical `Class` objects.
- All of the interface types must be visible by name through the specified class loader. In other words, for class loader `cl` and every interface `i`, the following expression must be true:

  ```
  Class.forName(i.getName(), false, cl) == i
  ```

- All of the types referenced by all public method signatures of the specified interfaces and those inherited by their superinterfaces must be visible by name through the specified class loader.
- All non-public interfaces must be in the same package and module, defined by the specified class loader and the module of the non-public interfaces can access all of the interface types; otherwise, it would not be possible for the proxy class to implement all of the interfaces, regardless of what package it is defined in.
- For any set of member methods of the specified interfaces that have the same signature:
  - If the return type of any of the methods is a primitive type or void, then all of the methods must have that same return type.
  - Otherwise, one of the methods must have a return type that is assignable to all of the return types of the rest of the methods.
- The resulting proxy class must not exceed any limits imposed on classes by the virtual machine. For example, the VM may limit the number of interfaces that a class may implement to 65535; in that case, the size of the `interfaces` array must not exceed 65535.

Note that the order of the specified proxy interfaces is significant: two requests for a proxy class with the same combination of interfaces but in a different order will result in two distinct proxy classes.

Parameters:

`loader` - the class loader to define the proxy class, may be `null` to represent the bootstrap class loader

`interfaces` - the list of interfaces for the proxy class to implement

`h` - the invocation handler to dispatch method invocations to

Returns:

a proxy instance with the specified invocation handler of a proxy class that is defined by the specified class loader and that implements the specified interfaces

Throws:

`IllegalArgumentException` - if any of the restrictions on the parameters are violated

NullPointerException - if the interfaces array argument or any of its elements are null, or if the invocation handler, h, is null

See Also:

Package and Module Membership of Proxy Class

isProxyClass

```
public static boolean isProxyClass(Class<?> cl)
```

Returns true if the given class is a proxy class.

Implementation Note:

The reliability of this method is important for the ability to use it to make security decisions, so its implementation should not just test if the class in question extends Proxy.

Parameters:

cl - the class to test

Returns:

true if the class is a proxy class and false otherwise

Throws:

NullPointerException - if cl is null

getInvocationHandler

```
public static InvocationHandler getInvocationHandler(Object proxy)
                                                     throws
IllegalArgumentException
```

Returns the invocation handler for the specified proxy instance.

Parameters:

proxy - the proxy instance to return the invocation handler for

Returns:

the invocation handler for the proxy instance

Throws:

IllegalArgumentException - if the argument is not a proxy instance

NullPointerException - if proxy is null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.